

How Data Moves Through the System

A walkthrough of what actually happens to a trip record, from the moment it's created to the moment someone queries it — and what's automatic vs. what still needs a person, as of 21 September 2026.

1

Source database

Working

Trip and camera events get written to a Postgres database as they happen — this is the day-to-day operational system, not part of the lake itself.

Right now: a realistic practice database (`mtsai-api-sim`) stands in for the real one, since a connection to the actual production database doesn't exist yet. Same shape, same behavior, synthetic data.

2

Export process

Automatic

Once a night, a small program wakes up on its own, connects to the database, and asks for everything that happened "yesterday" — one full day of rows.

How the data actually moves: it doesn't load the whole day into memory at once — it streams the rows out in batches using a database cursor, the same technique used for reports too large to pull in one go. Each batch gets converted into a compact, column-oriented file format (Parquet) as it comes in.

3

Cloud storage

Automatic

Those Parquet files get uploaded to cloud storage (S3) — cheap, durable, and organized so a specific day's data can be found without scanning everything.

Where it lands: a `raw/` area holds an unmodified, audit-grade copy of exactly what was in the source database that day — nothing is cleaned or changed here on purpose, so it can always be trusted as the original record.

4

Made queryable

Automatic

Uploaded files are just files until something tells the query engine they exist. This step registers the new day's data into a table that people can run plain SQL against.

Under the hood: the files are briefly registered as a temporary table, then merged into the permanent one with a database-style insert. If that same day ever gets re-processed, the old copy is swapped out first — so re-runs replace, they never duplicate.

5

Check & log

Automatic

Before calling the night's work done, the system checks its own results — did the number of rows that landed in the lake match what was actually read from the source?

The paper trail: a small receipt file gets written recording exactly what happened — row count, a checksum, how long it took, success or failure. If anything doesn't match, the run is marked failed and an alert fires rather than silently moving on.

6

Clean-up / organize step

Automatic

About 20 minutes after the export finishes, a second, separate process copies that day's data from the raw area into a second, tidier version meant for actual analysis.

Why a separate step: keeping "the exact original" and "the easy-to-use version" as two different copies means analysts get something clean to work with, while auditors always still have an untouched original to check against.

Analytics team

runs reports, only sees the clean copy

Forecasting team

predicts demand, only sees the clean copy

Auditors

can see both the raw and clean copies

Each team has its own separate login with its own permissions — tested by actually trying to break in as each one, not just reading the configuration.

● Built & working, on its own schedule ● Partly done ● Not built yet

What's Automatic Today vs. What's Not

Step	Status	What that means
1. Data exists in the source database	Working	Real for testing purposes; the actual production database isn't connected yet
2. Export process runs	Automatic	Runs on its own every night — confirmed by watching it fire without anyone starting it
3-5. Storage, cataloging, checking, logging	Automatic	Fully automatic, no manual work needed
6. Clean-up / organize step	Automatic	Runs on its own every night, 20 minutes after step 2 — people no longer only see raw data
Teams querying the data	Working	The access rules (who can see what) are built and tested
Catching up on older data	Partly done	A "process many days at once" mode exists and has been run for real over a multi-week range; hasn't been run over the full available history yet
Deleting one person's data on request	Working	Rehearsed twice against real data — removed from every table, and made unrecoverable, not just hidden
Weekly storage clean-up	Automatic	Runs every Sunday, tidies up old file versions and publishes storage-size numbers
Cost visibility & alerts	Working	A live dashboard plus email alerts if a nightly run fails or spending crosses a threshold

A note on "automatic": every step above that says "Automatic" has been directly observed firing on its own schedule at least once — not just configured and assumed to work. The evidence for that is in the companion Build Report PDFs.

Where to Actually See Each Step Happening

All in the AWS Console, region **ap-south-1**, account **690293068614**.

Step	Where to look
1. Source database	RDS → Databases → <code>mtsai-api-sim-test</code>
2. Export — the schedule itself	EventBridge → Scheduler → <code>mtsai-datalake-test-nightly-export</code>
2. Export — while it's running	ECS → Clusters → <code>mtsai-datalake-test</code> → Tasks tab (only visible for about an hour after it finishes)
2. Export — after the fact (reliable)	CloudWatch → Log groups → <code>/mtsai-datalake/test/export</code> → streams starting with <code>export/</code>
3. Cloud storage	S3 → <code>mtsai-datalake-test-690293068614-ap-south-1</code> → <code>raw/</code> folder

4. Made queryable	Athena → Query editor → database <code>test_raw</code> → <code>SELECT * FROM trip_events LIMIT 10</code>
5. Check & log — the receipts	Same S3 bucket → <code>export-manifests/</code> folder (one file per run)
6. Clean-up — the schedule	EventBridge → Scheduler → <code>mtsai-datalake-test-nightly-curate</code>
6. Clean-up — the result	S3 → <code>curated/</code> folder, or Athena → database <code>test_curated</code>
6. Clean-up — the logs	Same log group, streams starting with <code>curate/</code>
Weekly storage clean-up	Same log group, streams starting with <code>compact/</code>
Deleting a person's data	Same log group, streams starting with <code>erasure/</code> , plus <code>export-manifests/erasure/</code> in S3
Costs and alerts	CloudWatch → Dashboards → <code>mtsai-datalake-test-cost</code>

Generated from the current state of the mtsai-datalake project, 21 September 2026. Reflects what's actually built and tested, not the eventual full design. Companion to `MTSAi-Data-Lake-Build-Report.pdf` and `MTSAi-Data-Lake-Phase3-Build-Report.pdf` (technical detail).